



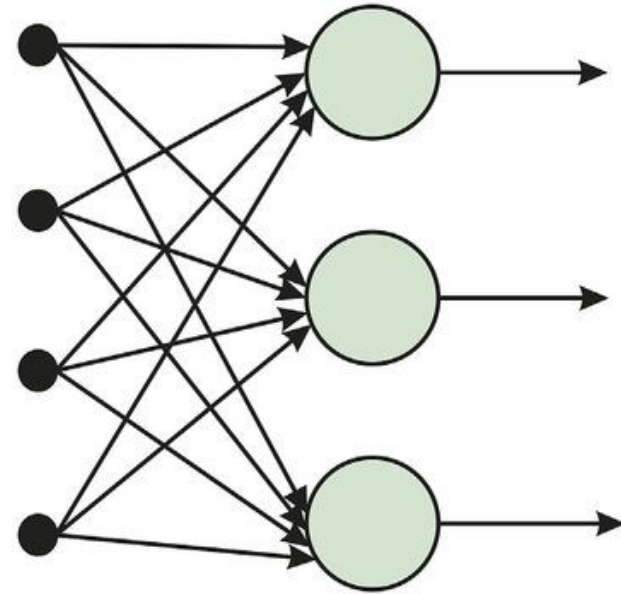
Recurrent Neural Networks (RNNs)



Recurrent Neural Networks (RNN)

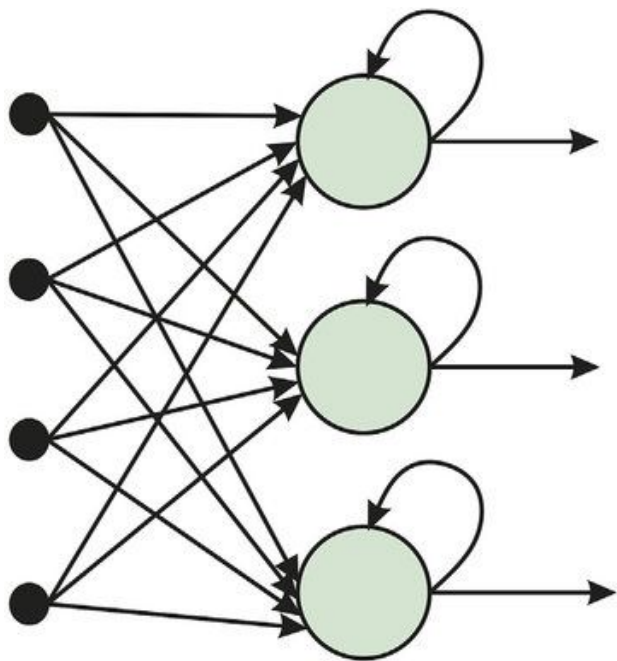
- Used for handling sequences of data.
 - Time series.
 - Text.
 - Speech.

Feed-Forward NN

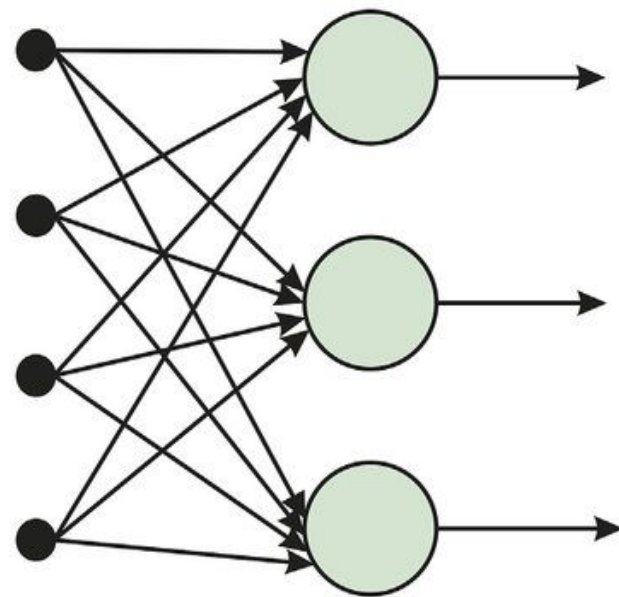


(b) Feed-Forward Neural Network

Recurrent NN

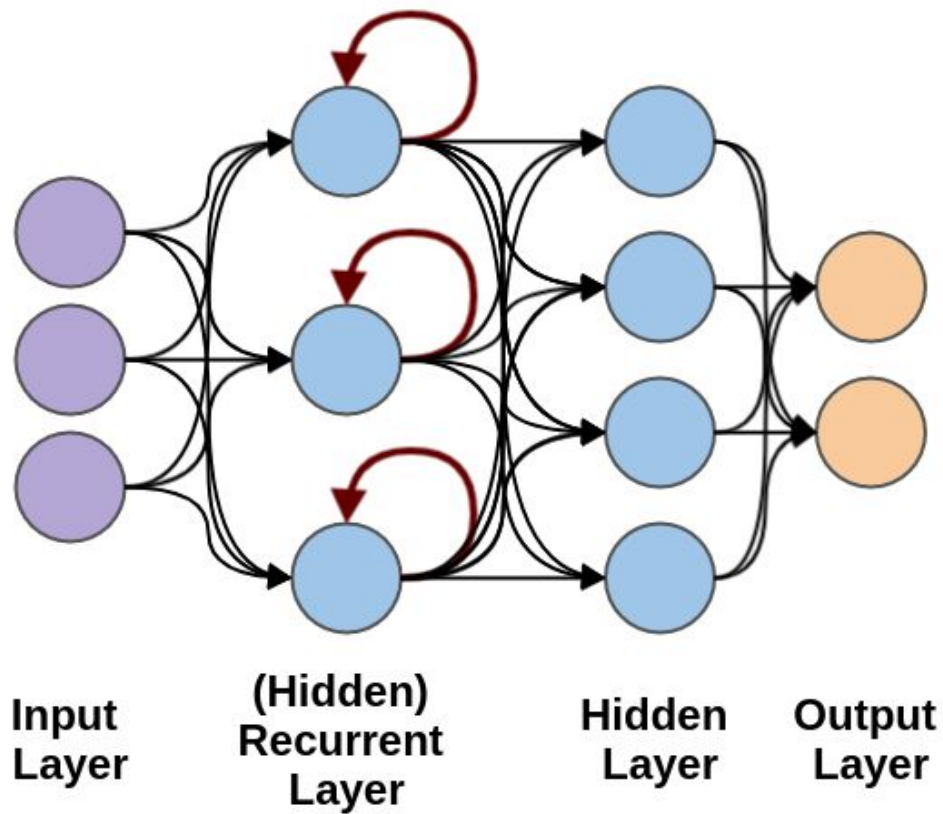


(a) Recurrent Neural Network

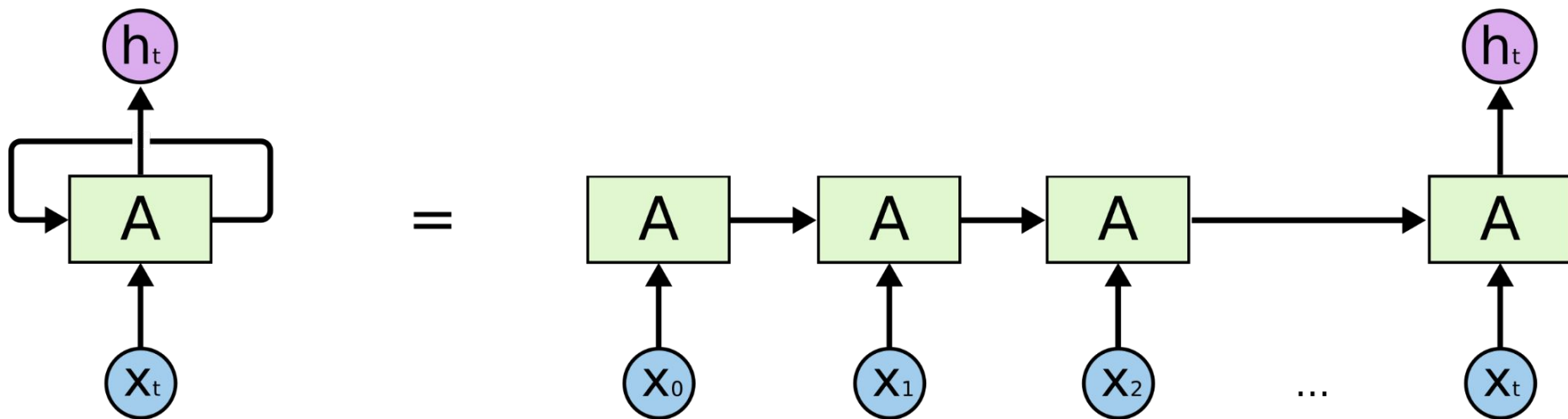


(b) Feed-Forward Neural Network

Recurrent NN



Recurrent NN



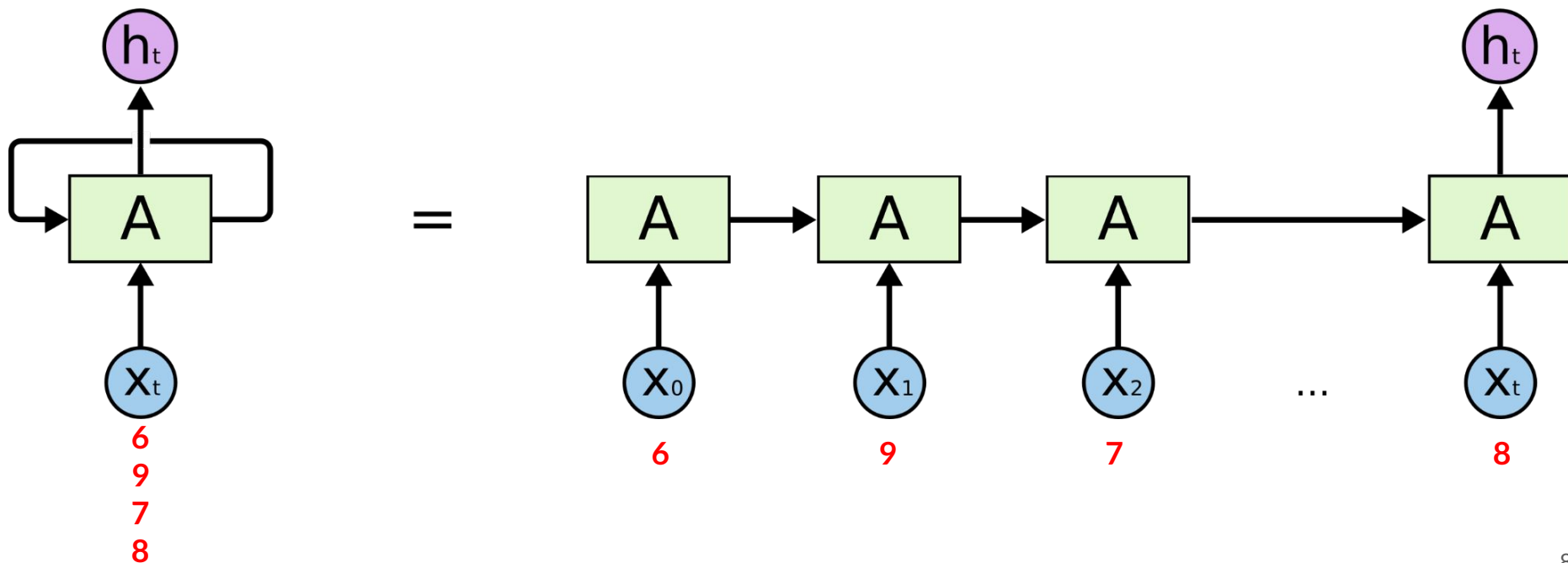


Recurrent NN

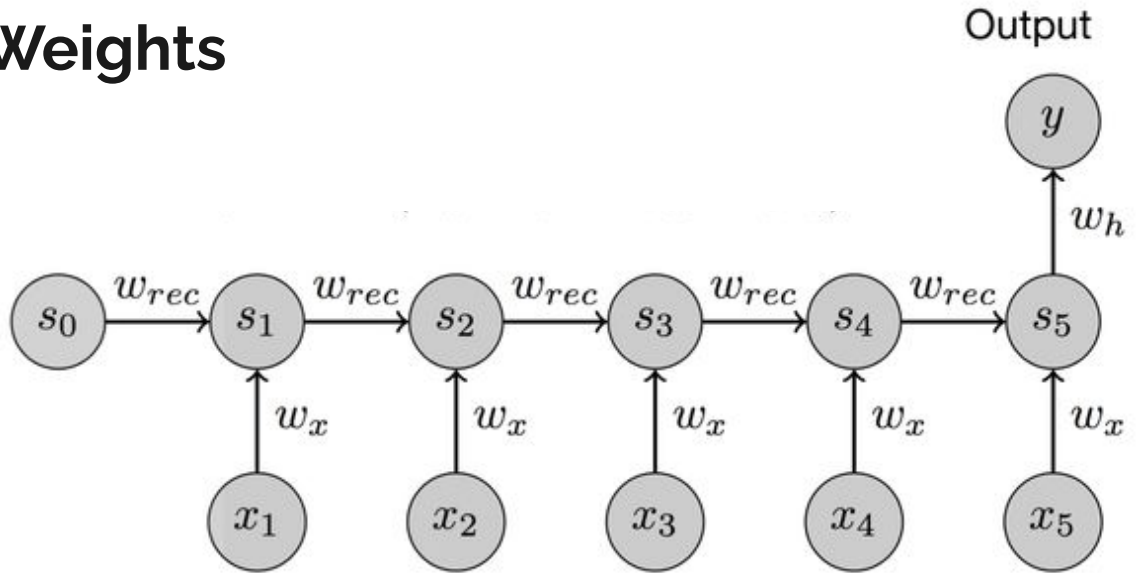
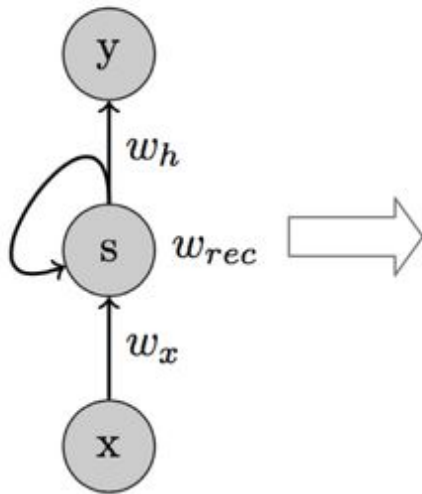
Sequence size = 4

	Today's temperature	Tomorrow's temperature
25/11/22	6	9
26/11/22	9	7
27/11/22	7	8
28/11/22	8	NaN

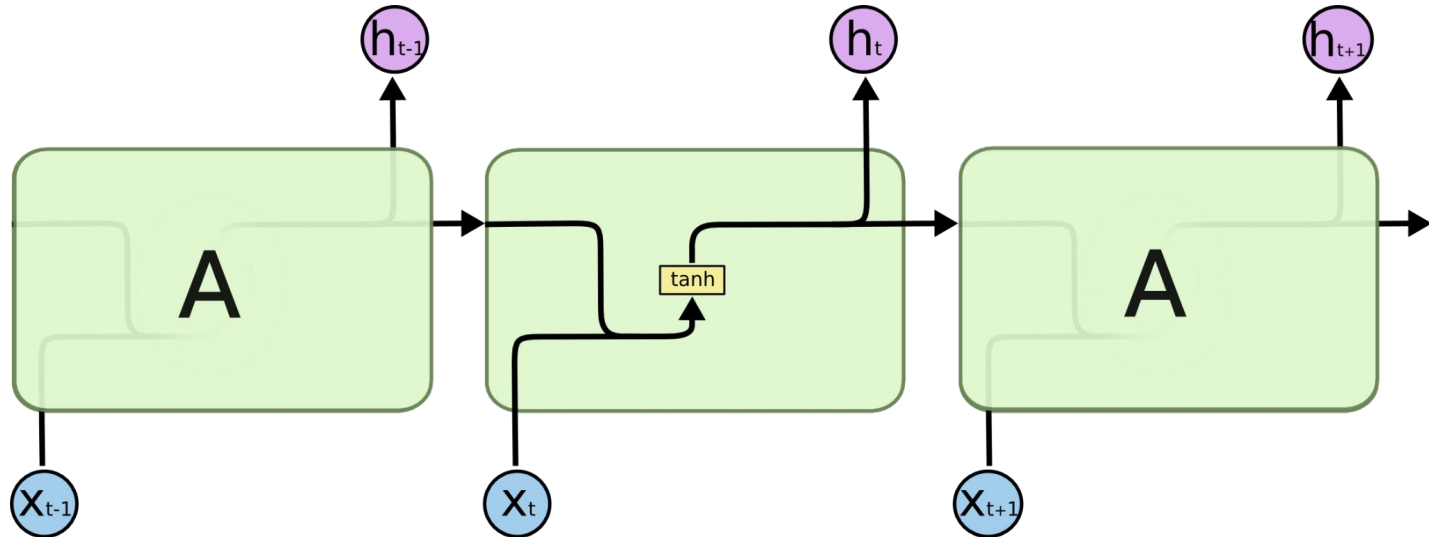
Recurrent NN – One output every t timesteps



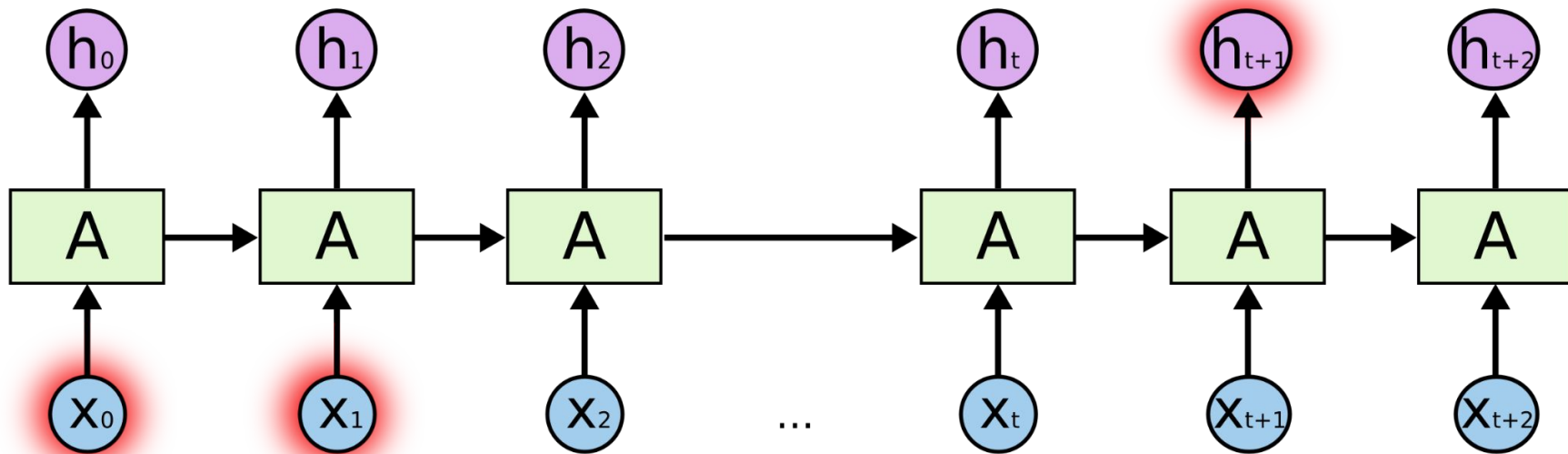
Recurrent NN – Weights



Recurrent NN – One output per timestep



Recurrent NN





Recurrent NN

- Use Backpropagation Through Time (BPTT).



Recurrent NN

- Use Backpropagation Through Time (BPTT).
- Pros:
 - It can model non-linear sequential relationships.



Recurrent NN

- Use Backpropagation Through Time (BPTT).
 - Pros:
 - It can model non-linear sequential relationships.
 - Cons:
 - Exploding gradients (when $w > 1$).
 - Vanishing gradients (when $w < 1$).
 - Not well suited for long temporal data.
- Use **tanh** activation function.



Long Short-Term Memory (LSTM)

- It is a RNN.
- Introduced in 1997.
- Deals with vanishing gradients.
- Can “store” information for long periods.



Long Short-Term Memory (LSTM)

- 3 gates regulate the flow of information through time:
 - **Forget gate:** Decides what information to discard or to keep in “long-term memory”.



Long Short-Term Memory (LSTM)

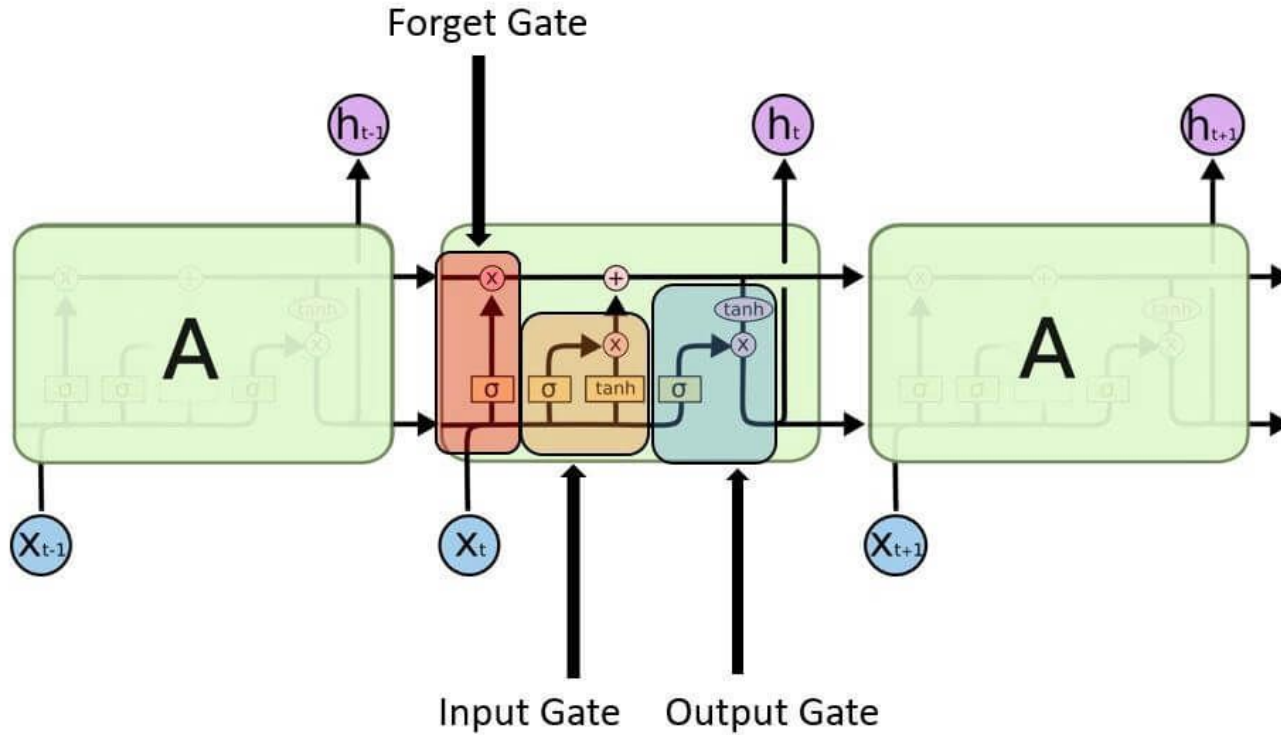
- 3 gates regulate the flow of information through time:
 - **Forget gate:** Decides what information to discard or to keep in “long-term memory”.
 - **Input gate:** Decides what new information to store.



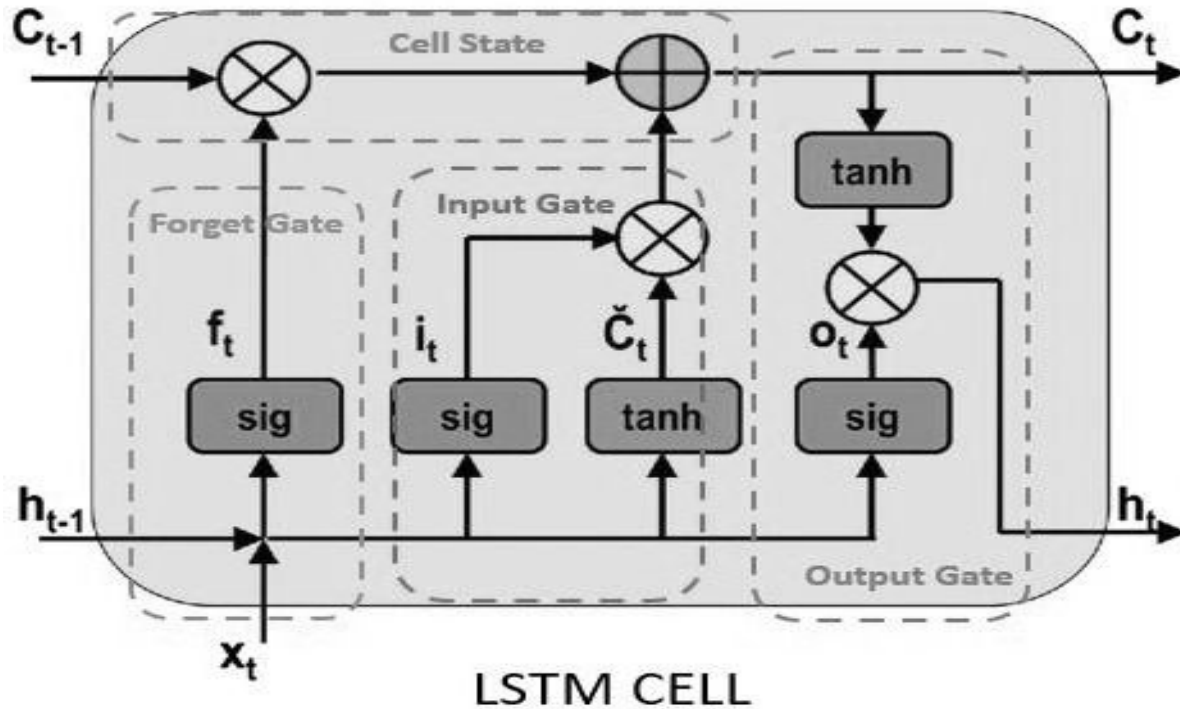
Long Short-Term Memory (LSTM)

- 3 gates regulate the flow of information through time:
 - **Forget gate:** Decides what information to discard or to keep in “long-term memory”.
 - **Input gate:** Decides what new information to store.
 - **Output gate:** Decides what information to discard or to keep in “short-term memory”.

LSTM Cell



Long Short-Term Memory (LSTM)





LSTM

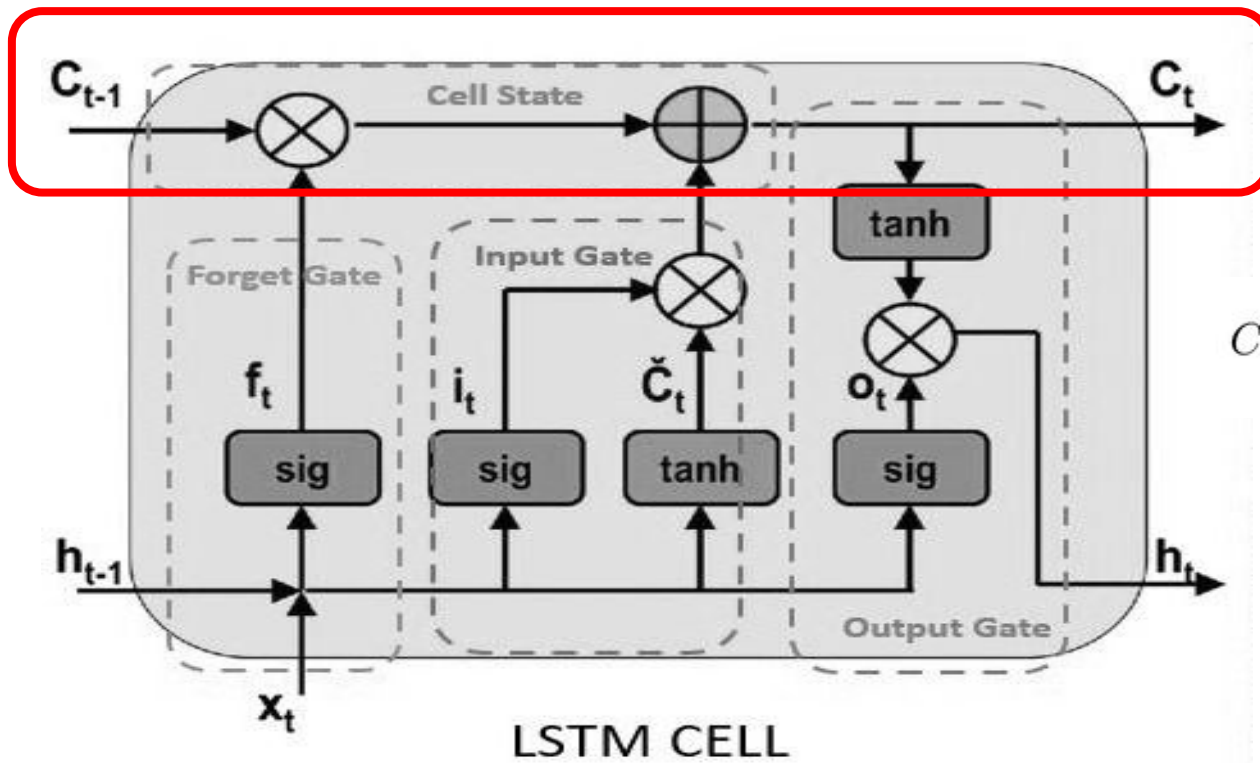
- Cell state (C) and hidden state (h) are vectors of N units (hyperparameter).
- These vectors are an abstraction of the “memory”.
- The output of each gate is a vector of length N .
- The **sigmoid** and **tanh** are applied element-wise to each state vector.
 - In this way, the network applies a selective operation to the cell and hidden states.
 - E.g., selective forgetting.
 - See forget gate example in a few slides.



LSTM

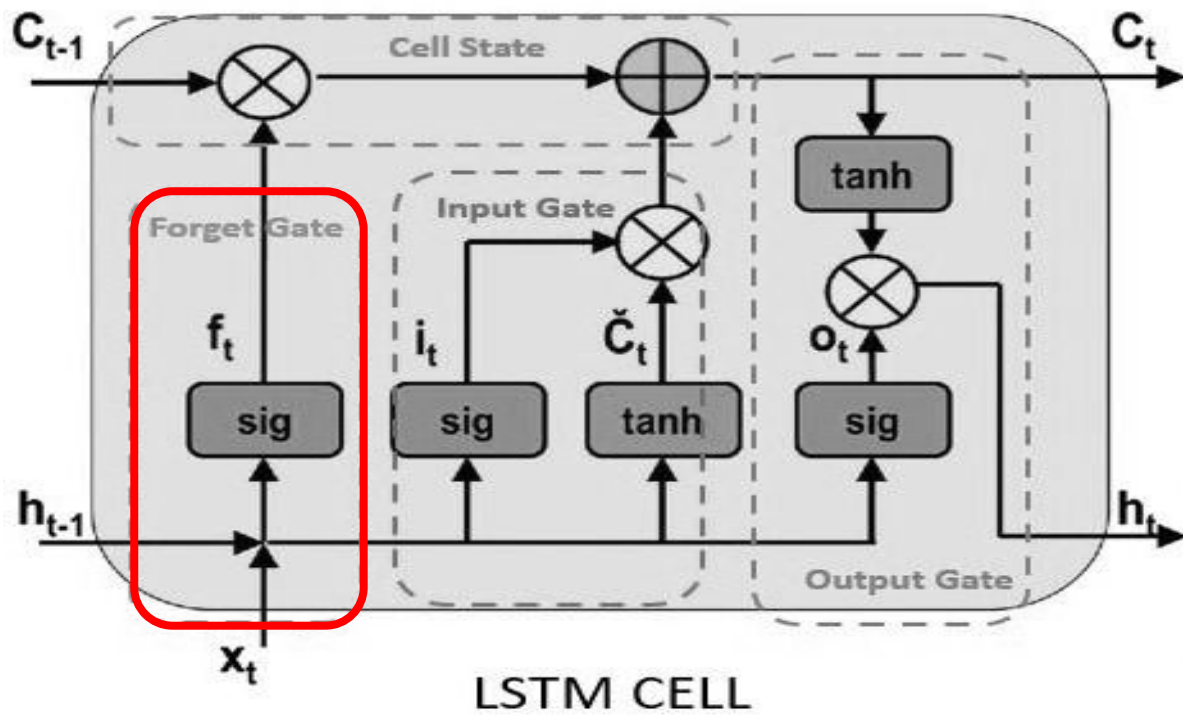
- Gates' internal structure:
 - 1 NN for the Forget Gate.
 - 2 NN for the Input Gate (Input + Candidate).
 - 1 NN for the Output Gate.

LSTM



$$C_t = f_t * C_{t-1} + i_t * \tilde{C}_t$$

LSTM



$$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$$



LSTM – Forget Gate Example

- Suppose we want to predict the temperature for the next day.
- Suppose a state of length 2 (i.e., 2 numbers that, abstractly, define each state).
- The cell state learns to store selective long-term information in just 2 numbers, for example:
 - Seasonal trend: Is it summer or winter? ($C_{t-1}[0]$).
 - Temperature change: Has it been getting warmer or cooler recently? ($C_{t-1}[1]$).



LSTM – Forget Gate Example

- At time step t , the input is today's weather data (x_t), including features like:
 - Today's temperature
 - Humidity
 - Wind speed
 - Rainfall



LSTM – Forget Gate Example

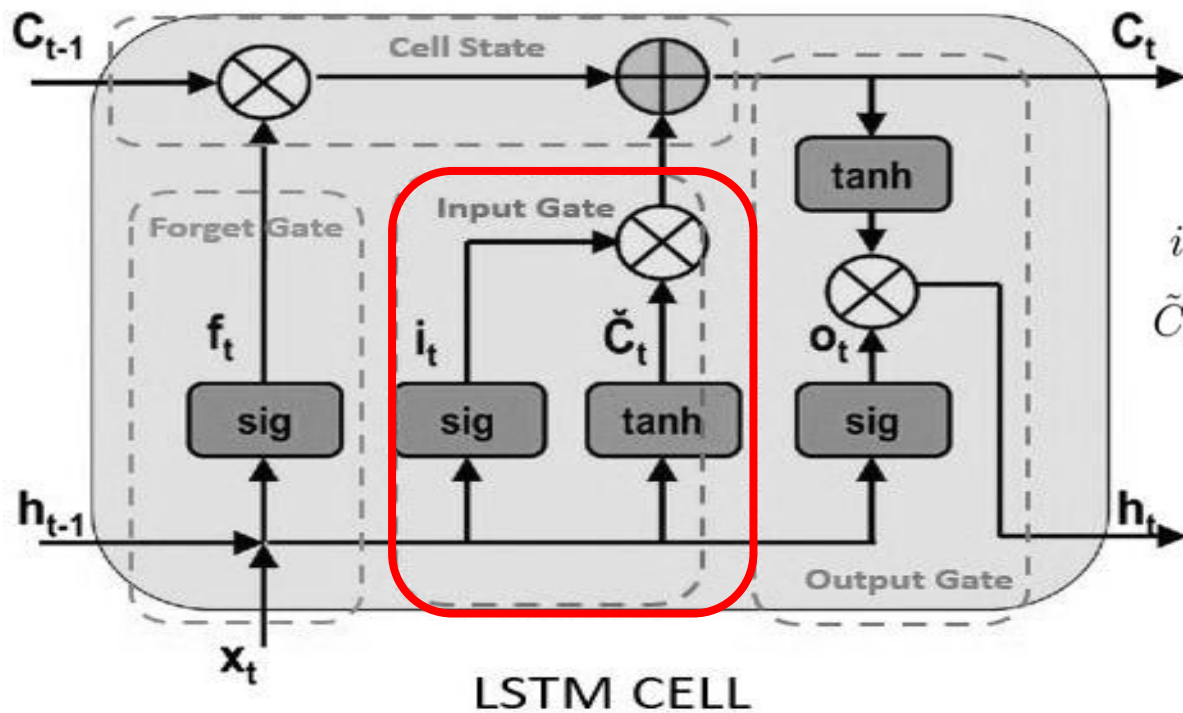
- The forget gate decides, based on the current data x_t , which info. to keep in the cell state.
- Imagine it gives the following output: $f_t = [0.8, 0.3]$. This means:
 - Retain 80% of the "seasonal trend" information.
 - E.g., it's summer, so high temperatures are expected.



LSTM – Forget Gate Example

- The forget gate decides, based on the current data x_t , which info. to keep in the cell state.
- Imagine it gives the following output: $f_t = [0.8, 0.3]$. This means:
 - Retain 80% of the "seasonal trend" information.
 - E.g., it's summer, so high temperatures are expected.
 - Retain only 30% of the "temperature change" information.
 - E.g., a sudden shift in temperature occurred, and the model decides recent changes are less relevant compared to the current input.

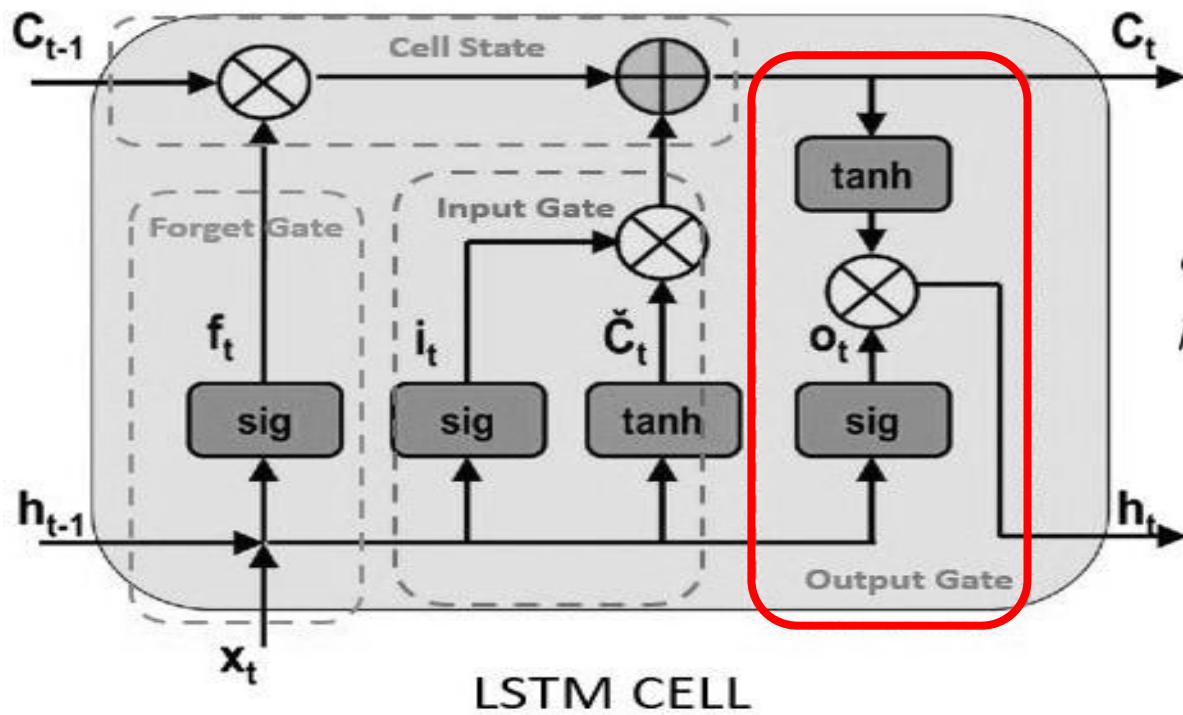
LSTM

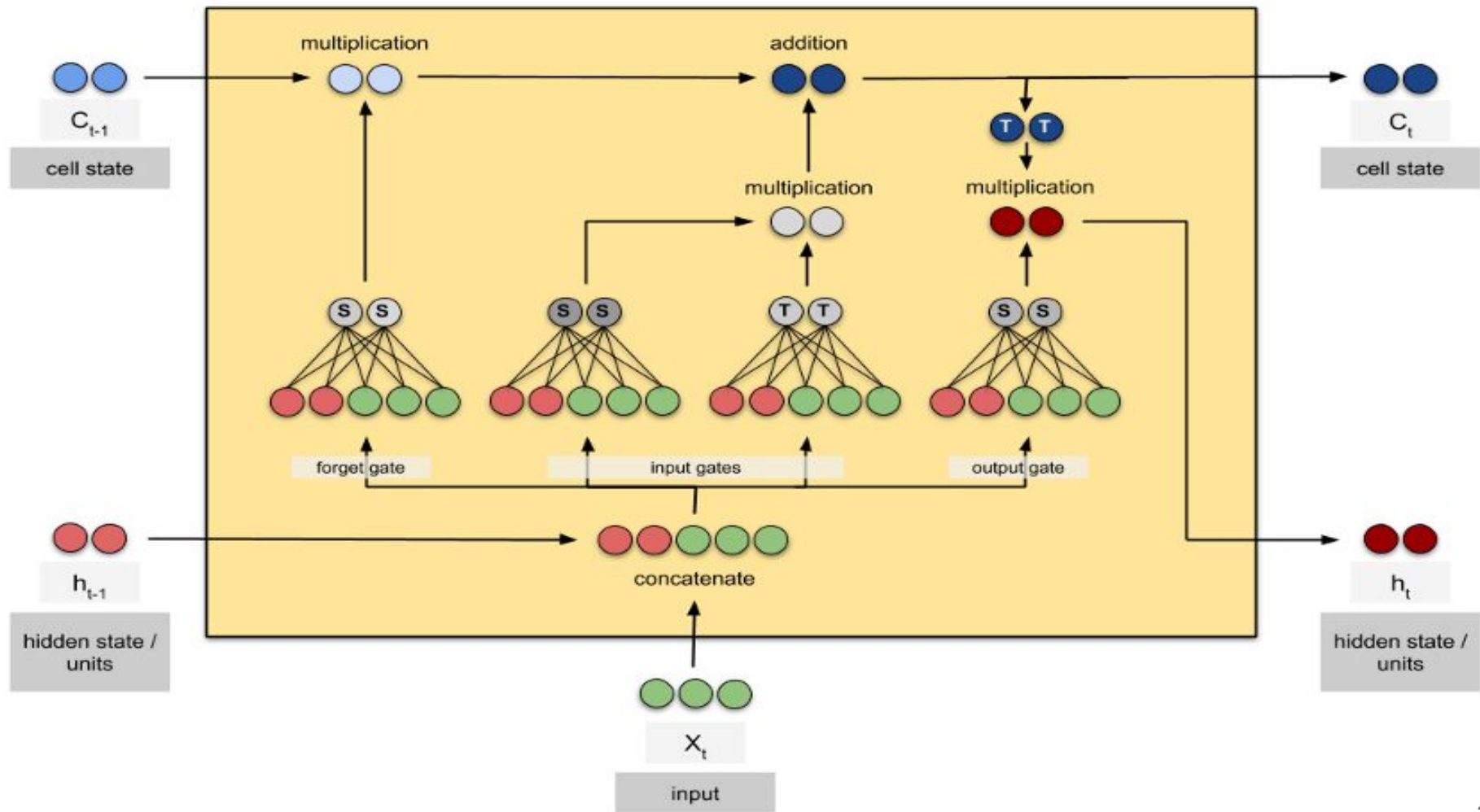


$$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$$

$$\tilde{C}_t = \tanh(W_C \cdot [h_{t-1}, x_t] + b_C)$$

LSTM



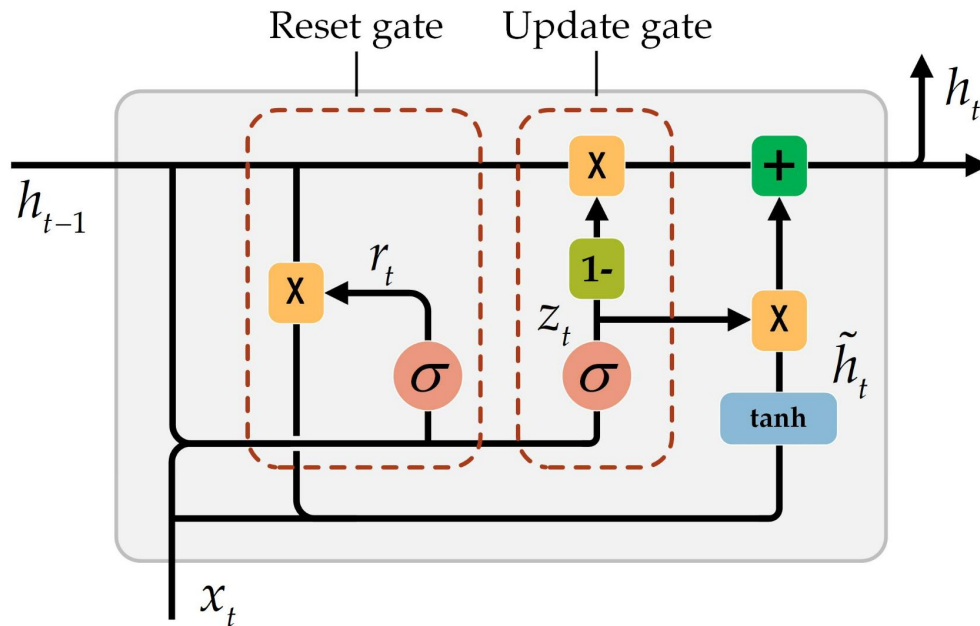




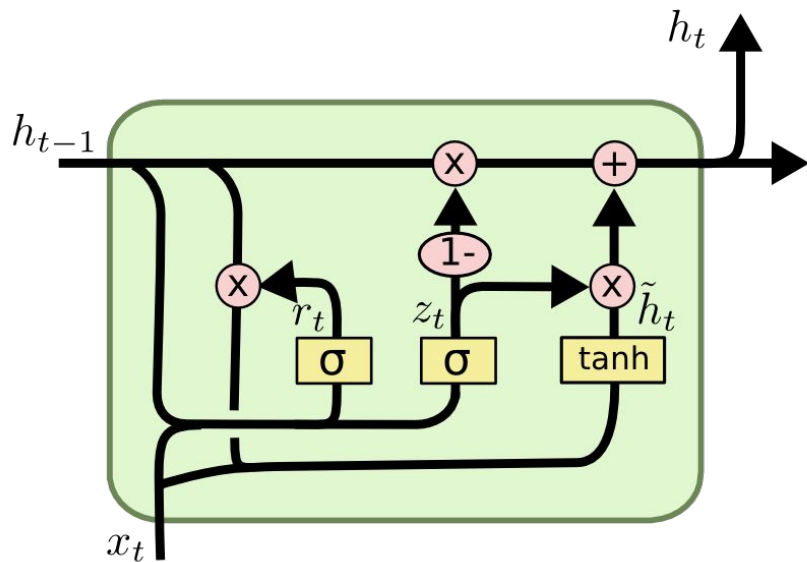
Gated Recurrent Unit (GRU)

- It is an RNN.
- Introduced in 2014.
- It is a “simplified LSTM”.
- Composed of 2 gates:
 - **Reset gate:** Decides what information to discard or to keep.
 - **Update gate:** Decides what new information to store.

GRU Cell



Gated Recurrent Unit (GRU)



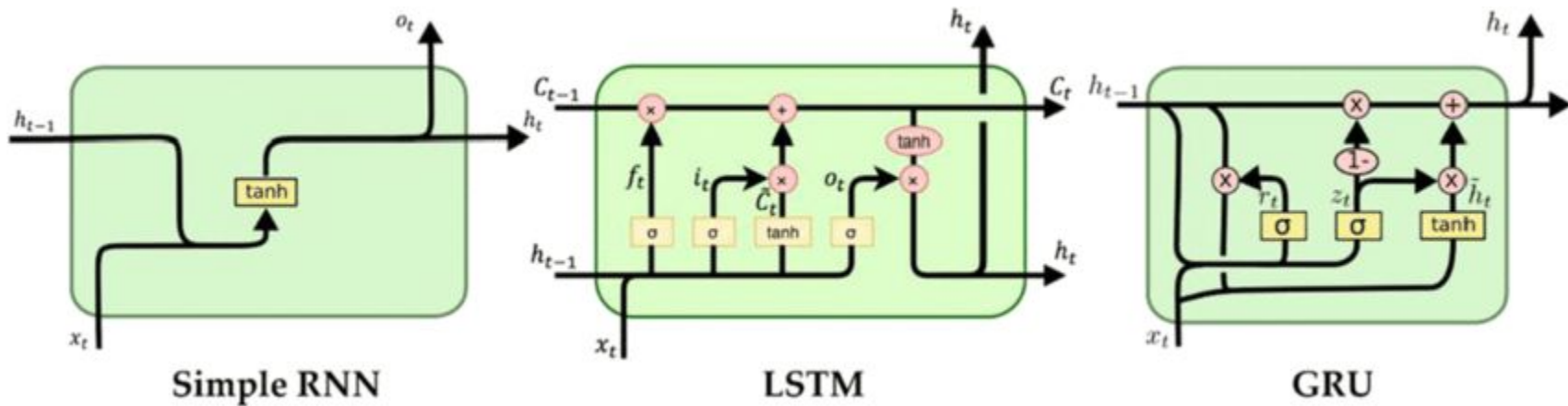
$$z_t = \sigma(W_z \cdot [h_{t-1}, x_t])$$

$$r_t = \sigma(W_r \cdot [h_{t-1}, x_t])$$

$$\tilde{h}_t = \tanh(W \cdot [r_t * h_{t-1}, x_t])$$

$$h_t = (1 - z_t) * h_{t-1} + z_t * \tilde{h}_t$$

RNN vs LSTM vs GRU





RNN vs LSTM vs GRU

- RNN
 - Generally not recommended for long sequences.
 - Vanishing/Exploding gradients.



RNN vs LSTM vs GRU

- LSTM
 - Normally performs better than simple RNN.
 - Good for long sequences.
 - Solves vanishing gradient problem.
 - Keeps exploding gradient problem.



RNN vs LSTM vs GRU

- GRU
 - Similar performance as LSTM.
 - Less parameters to train (i.e., easier and faster).
 - Less memory.
 - LSTMs may work better in very long sequences.



RNNs in Keras

- From tensorflow.keras.layers
 - SimpleRNN
 - LSTM
 - GRU

Typical Sklearn Input

	Temperature (X1)	Humidity (X2)
25/11/22	6	92
26/11/22	9	85
27/11/22	7	88
28/11/22	8	91
29/11/22	6	89



Shape: (5, 2)

X = [
[6, 92],
[9, 85],
[7, 88],
[8, 91],
[6, 89],
]

Typical Sklearn Input

	Temperature (X1)	Humidity (X2)
25/11/22	6	92
26/11/22	9	85
27/11/22	7	88
28/11/22	8	91
29/11/22	6	89



```
X = [  
    [6, 92],  
    [9, 85],  
    [7, 88],  
    [8, 91],  
    [6, 89],  
]
```

Not working for
keras RNNs



RNNs Keras Input

	Temperature (X1)	Humidity (X2)
25/11/22	6	92
26/11/22	9	85
27/11/22	7	88
28/11/22	8	91
29/11/22	6	89

Seq. 1

Decide: Sequence length.
E.g., 3

The model learns how the patterns from these
3 time steps can predict the next value.



RNNs Keras Input

	Temperature (X1)	Humidity (X2)
25/11/22	6	92
26/11/22	9	85
27/11/22	7	88
28/11/22	8	91
29/11/22	6	89

Seq. 2

Decide: Sequence length.
E.g., 3

The model learns how the patterns from these
3 time steps can predict the next value.



RNNs Keras Input

	Temperature (X1)	Humidity (X2)
25/11/22	6	92
26/11/22	9	85
27/11/22	7	88
28/11/22	8	91
29/11/22	6	89

Seq. 3

Decide: Sequence length.
E.g., 3

The model learns how the patterns from these
3 time steps can predict the next value.

RNNs Keras Input

	Temperature (X1)	Humidity (X2)
25/11/22	6	92
26/11/22	9	85
27/11/22	7	88
28/11/22	8	91
29/11/22	6	89



Shape: (3, 3, 2)

$X = \begin{bmatrix} \begin{bmatrix} 6, 92 \end{bmatrix}, \\ \begin{bmatrix} 9, 85 \end{bmatrix}, \\ \begin{bmatrix} 7, 88 \end{bmatrix}, \\ \begin{bmatrix} 9, 85 \end{bmatrix}, \\ \begin{bmatrix} 7, 88 \end{bmatrix}, \\ \begin{bmatrix} 8, 91 \end{bmatrix}, \\ \begin{bmatrix} 7, 88 \end{bmatrix}, \\ \begin{bmatrix} 8, 91 \end{bmatrix}, \\ \begin{bmatrix} 6, 89 \end{bmatrix} \end{bmatrix}$



Resources

- <https://colah.github.io/posts/2015-08-Understanding-LSTMs/>
- <https://www.kaggle.com/code/kmkarakaya/lstm-understanding-the-number-of-parameters>
- <https://www.youtube.com/watch?v=YCzL96nL7j0>
- <https://towardsdatascience.com/understanding-gru-networks-2ef37df6c9be>